

Taller Investigativo

Explorando Google Colab y su relación con la programación en Python

Maycol Slon Buitrago Manrique

Oscar Mahecha Camargo

Ingeniería de sistemas, Corporación Universitaria Remington

Compiladores

2026

Tabla de contenido

Preguntas de investigación.....	3
1 ¿Qué es Google Colab?.....	3
2 ¿Cuáles son las principales características de Google Colab?.....	4
3 ¿Qué ventajas tiene Google Colab frente a instalar Python en un computador local?	5
4 ¿Cómo está estructurado un entorno de trabajo en Google Colab?	6
4.1 Celdas de código	6
4.2 Celdas de texto	6
4.3 Menús.....	6
4.4 Ejecución del código.....	7
5 Relación entre el léxico y la programación en Python dentro de Google Colab.	7
5.1 Ejemplos	7
6 Relación entre la sintaxis y la escritura de código en Python en Google Colab.....	9
6.1 Código correcto.....	9
6.2 Código con errores	9
7 ¿Cómo se relaciona la gramática de un lenguaje de programación con Python?	10
7.1 Cómo organiza Python las instrucciones:	10
7.2 Prueba en Colab	11
8 La importancia de la indentación en Python dentro de Google Colab.	11
8.1 Un código funcione correctamente	12
8.2 Otro tenga error por mala indentación.	12
9 Aplicaciones prácticas de Google Colab en proyectos reales.....	13
10 Reflexión crítica.....	14
11 Fuentes	16

Preguntas de investigación

1 ¿Qué es Google Colab?

Explica qué es esta herramienta, quién la desarrolló y para qué se utiliza en el aprendizaje y desarrollo de programación.

Google Colab, es un entorno gratuito de Google que nos permite escribir y ejecutar código de python desde el propio navegador y no requiere ningún tipo de instalación. Esta herramienta funciona con cuadernos de Jupyter, y en estos se puede crear y compartir documentos con código en vivo, siendo muy practico para trabajos grupales. Además, es posible incluir en los proyectos texto explicativo, imágenes y gráficos.

La maquina virtual es la que se encarga de correr el código, no el computador de la persona que lo ejecuta, entonces es muy práctico porque puede funcionar con equipos con pocos recursos. Toda la información se almacena en Google drive y puede compartirse como cualquier documento de Google.

Fue desarrollada por Google Research, un equipo de desarrollo de Google que buscaba facilitar la investigación y la educación en este tipo de entornos, puesto que el objetivo era incentivar su uso practico en conjunto con otras habilidades como el machine learning.

Para que se usa en el aprendizaje y desarrollo:

Ayuda a aprender a programar porque también sirve como un entorno donde se puede ir ejecutando código por partes y visualizar el resultado, también para el análisis de datos y la realización de graficas, otro punto importante sería el entrenamiento de los modelos de IA y aprendizaje autónomo.

2 ¿Cuáles son las principales características de Google Colab?

Describe al menos cinco características importantes que faciliten el desarrollo de programas en Python.

- **No requiere instalación:** solo es necesario acceder al navegador y tener una cuenta de Google.
- **Es un entorno que ya se encuentra configurado para Python:** trae también las librerías más populares, ejemplo, NumPy, pandas, Matplotlib, scikit-learn, TensorFlow, PyTorch).
- **Es posible acelerar procesos por medio del hardware:** permite el uso de recursos como GPU y TPU, estos son demasiado útiles para todo lo que tiene que ver con inteligencia artificial, aunque en la versión gratuita están limitadas estas herramientas.
- **Integración con Google Drive:** allí se guardan los cuadernos o proyectos y también se pueden leer archivos que estén el drive.
- **Cuaderno interactivo:** se puede hacer uso tanto de las celdas de código como del texto ya sea para educación o documentar procesos.
- **Colaboración en tiempo real:** puede compartirse con acciones limitadas, es decir, es posible dar permiso para leer, comentar o editar.
- **Instalación de paquetes:** también pueden añadirse librerías adicionales de Python
- **Visualización integrada:** Se pueden visualizar de forma instantánea los gráficos y tablas trabajadas, estos aparecen debajo de las celdas.

3 ¿Qué ventajas tiene Google Colab frente a instalar Python en un computador local?

Analiza aspectos como acceso, colaboración, almacenamiento en la nube y recursos computacionales.

Aspecto	Google Colab	Python local
Acceso	Puede ser desde cualquier dispositivo con internet y contar con cuenta en Google.	Solo se accede en el equipo donde se instaló
Configuración	No hay que configurar nada solo se abre y se empieza a codificar.	Es necesario Instalar Python, pip, el editor y en muchos casos entornos virtuales
Colaboración	Es posible Compartir por medio de enlace, comentar y editar en grupo de trabajo	Toca Enviar archivos o usar Git, despliegue.
Almacenamiento	Se almacena en la nube en Drive y cuenta con respaldo automático	En local se almacena en el disco del equipo, muchas veces con riesgo de pérdida
Recursos	Cuentan GPU/TPU y mucha más memoria de la que tienen muchos portátiles en local	Solo puede trabajar con los recursos y limitaciones del hardware
Librerías	Viene con las librerías ya integradas y con las actualizaciones	Toca instalarlas y gestionarlas de forma manual
Compatibilidad	Debe funcionar desde cualquier punto accedido.	Acá si se es dependiente de versiones y sistema operativo

4 ¿Cómo está estructurado un entorno de trabajo en Google Colab?

Explica los elementos principales, por ejemplo:

4.1 Celdas de código

Es en estos espacios donde se escribe el código python y con shift + enter puede ejecutarse, también existe el botón de play.

El resultado aparecerá debajo de las celdas de código. Las variables que se definan en una celda quedan en disponibilidad siempre y cuando la sesión se mantenga activa.

4.2 Celdas de texto

En esta sección es posible agregar títulos, enlaces, imágenes, además sirven para la documentación y explicación del código.

4.3 Menús

1. Archivo: crear, abrir, guardar y descargar cuadernos
2. Editar: deshacer, copiar celdas, borrar salidas.
3. Ver: tabla de contenido, modo de visualización.
4. Insertar: agregar celdas de código o texto.
5. Entorno de ejecución: ejecutar todo, reiniciar sesión, cambiar el tipo de hardware (CPU, GPU, TPU).
6. Herramientas: configuración y atajos de teclado.
7. Ayuda: documentación y preguntas frecuentes.

4.4 Ejecución del código

Cuando se desea ejecutar el código, al momento de dar click en play el código se envía a un Kernel de Python que se encuentra alojado en la máquina virtual de Google, este es el que interpreta este código y devuelve el resultado de nuevo al navegador. En el cuaderno de colab el orden de ejecución de cada celda se determina por el número que se encuentra dentro de los corchetes, y es muy importante porque hay celdas que pueden depender de variables que existen en otras celdas.

5 Relación entre el léxico y la programación en Python dentro de Google Colab.

Explica qué es el léxico en programación y menciona ejemplos.

Léxico: representa todo ese conjunto de palabras o símbolos con significado que es reconocido por determinado lenguaje de programación. En el proceso de compilación, y también en los intérpretes, el análisis léxico es la primera fase, y es muy importante porque lee el código como si fuera un texto y empieza a dividirlo en tokens, y entonces cada token tiene un identificador de tipo y su respectivo valor.

5.1 Ejemplos

Categoría	Descripción	Ejemplos
Palabras reservadas	Poseen un significado fijo y no se utilizan como nombre de variables	if, else, for, while, def, return, class, import, True, None

Identificadores	Son los nombres que da la persona o la herramienta que codifica a las variables, funciones y clases.	edad, nombre_usuario, calcular_total
Literales	Son los valores que se escriben directamente en el código	25 es entero, 3.14 es decimal, "Hola" es cadena, True es un booleano
Operadores	Son los símbolos o palabras que realizan operaciones, estos pueden variar dependiendo del lenguaje.	+, -, *, /, ==, >=, and, or, not
Delimitadores	Ayudan a separar o agrupar elementos, además de ser la estructura de componentes específicos en Python como listas, diccionarios, tuplas, etc.	(), [], {}, ,, :

```

▶ import keyword
  print(keyword.kwlist)  # lista de palabras reservadas

import tokenize, io
codigo = "edad = 18\n"
for tok in tokenize.generate_tokens(io.StringIO(codigo).readline):
    print(tokenize.tok_name[tok.type], repr(tok.string))

... ['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await',
NAME 'edad'
OP '='
NUMBER '18'
NEWLINE '\n'
ENDMARKER '']

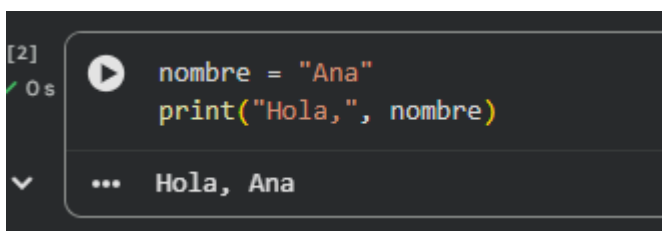
```

6 Relación entre la sintaxis y la escritura de código en Python en Google Colab.

Explica qué es la sintaxis y muestra un ejemplo de un código correcto y otro con error de sintaxis.

La sintaxis hace énfasis en las reglas que se deben aplicar para estructurar de forma adecuada todos esos componentes del léxico y así poder formar instrucciones válidas, estas reglas determinan cómo se van a combinar los tokens. En los procesos de compilación, en la parte de análisis sintáctico se verifica que las secuencias de tokens respeten las reglas gramaticales del lenguaje.

6.1 Código correcto

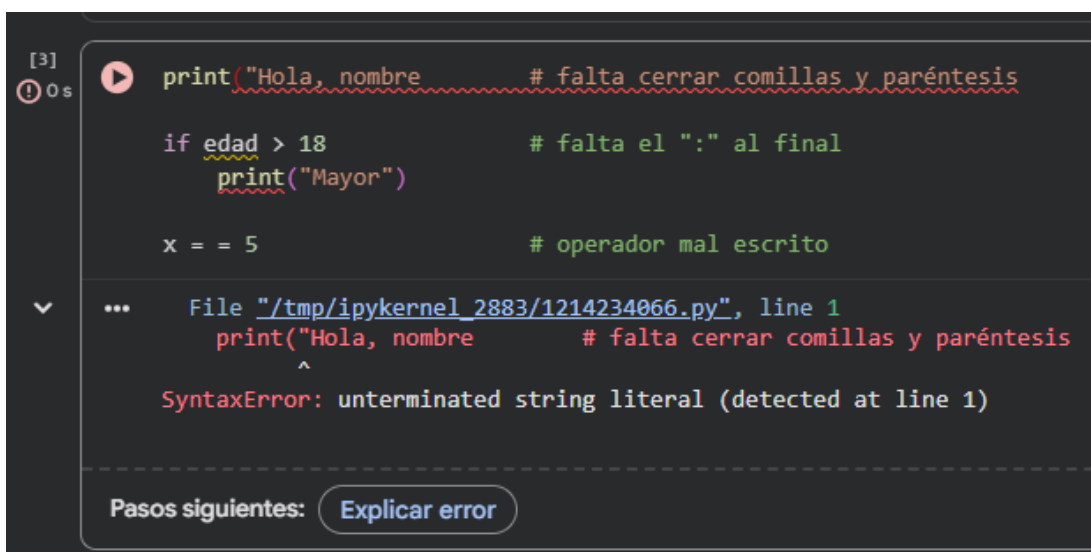


The screenshot shows a code cell in Google Colab. On the left, it says "[2]" and "0 s". A play button icon is next to the code. The code is:


```
nombre = "Ana"
print("Hola,", nombre)
```

 Below the code, the output is displayed: "... Hola, Ana".

6.2 Código con errores



The screenshot shows a code cell in Google Colab with syntax errors. On the left, it says "[3]" and "0 s" with a red error icon. A play button icon is next to the code. The code is:


```
print("Hola, nombre # falta cerrar comillas y paréntesis
if edad > 18 # falta el ":" al final
    print("Mayor")
x = = 5 # operador mal escrito
```

 Below the code, the error message is displayed:


```
... File "/tmp/ipykernel_2883/1214234066.py", line 1
      print("Hola, nombre # falta cerrar comillas y paréntesis
            ^
SyntaxError: unterminated string literal (detected at line 1)
```

 At the bottom, there is a section "Pasos siguientes:" with a button labeled "Explicar error".

7 ¿Cómo se relaciona la gramática de un lenguaje de programación con Python?

Explica cómo Python organiza las instrucciones para que el programa tenga sentido lógico y pueda ejecutarse correctamente.

La gramática son las reglas estructurales que determinan de forma exacta que programas son validos respecto a un lenguaje de programación. Para entenderlo mejor, la sintaxis hace parte de lo que el programador escribe y cumple; y la gramática es la especificación en forma matemática que lo describe, es decir, la ejecución interna, de las instrucciones dadas dependientes de una estructuración adecuada.

7.1 Cómo organiza Python las instrucciones:

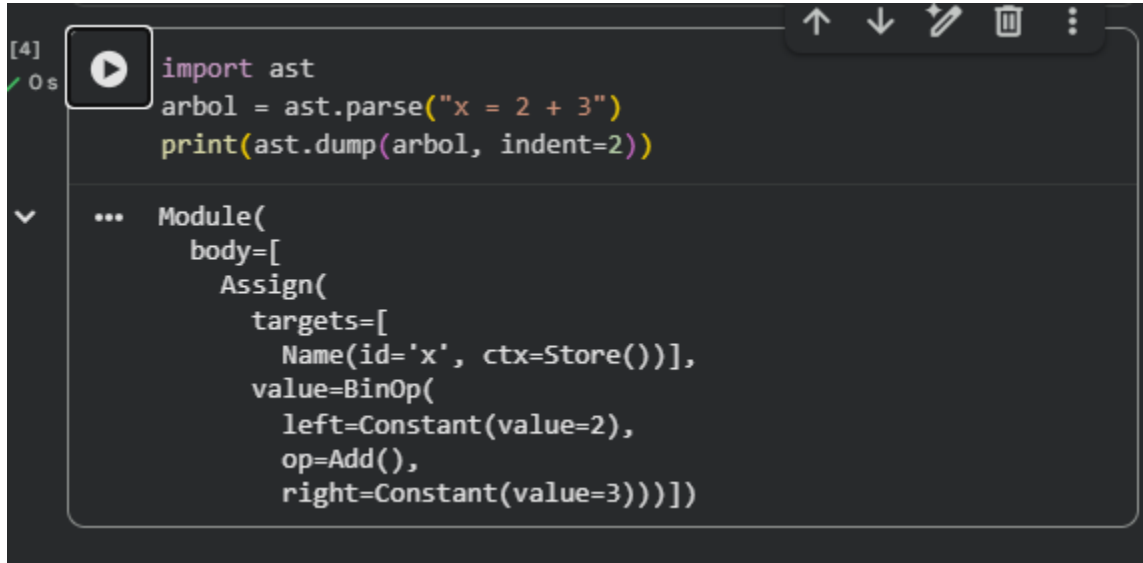
1. En la etapa de análisis léxico todo el texto, es decir, el código se va a convertir en tokens, incluyendo los tokens especiales INDENT y DEDENT que son los de indentacion.

2. Después del análisis lexico el analizador sintáctico (parser) aplica la gramática y posteriormente construye un árbol de sintaxis abstracta (AST), que es una estructura de datos en forma de árbol donde se representa la estructura lógica y también la jerarquía que existe en el código analizado.

3. El árbol se traduce a bytecode, es decir, código intermedio de bajo nivel que se diseña para poder ser ejecutado por la maquina virtual de Python.

Las instrucciones simples, asignaciones, print, return y sentencias compuestas, if, for, while, def, class se combinan en bloques jerárquicos, y entonces es esa jerarquía la que da sentido lógico al programa.

7.2 Prueba en Colab



```
[4] 0s  import ast
arbol = ast.parse("x = 2 + 3")
print(ast.dump(arbol, indent=2))
```

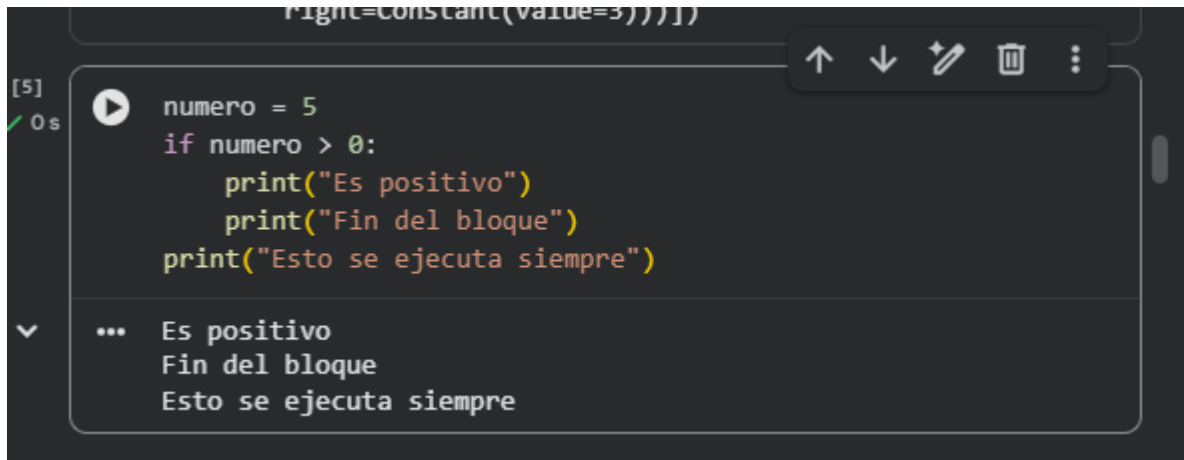
```
... Module(
  body=[
    Assign(
      targets=[
        Name(id='x', ctx=Store())],
      value=BinOp(
        left=Constant(value=2),
        op=Add(),
        right=Constant(value=3))))]
```

8 La importancia de la indentación en Python dentro de Google Colab.

Explica por qué la indentación es fundamental en Python y muestra un ejemplo donde un código funcione y otro no.

La indentación en Python es muy importante porque define la estructura del programa. En otros programas se usan delimitadores como llaves, en su lugar python usa los espacios, estos van al inicio de la línea y así es como se indica que instrucciones pertenecen a un bloque determinado como el if, for, una función, etc. En la etapa de análisis lexico se generan los tokens basándose en la estructura de indentación.

8.1 Un código funcione correctamente



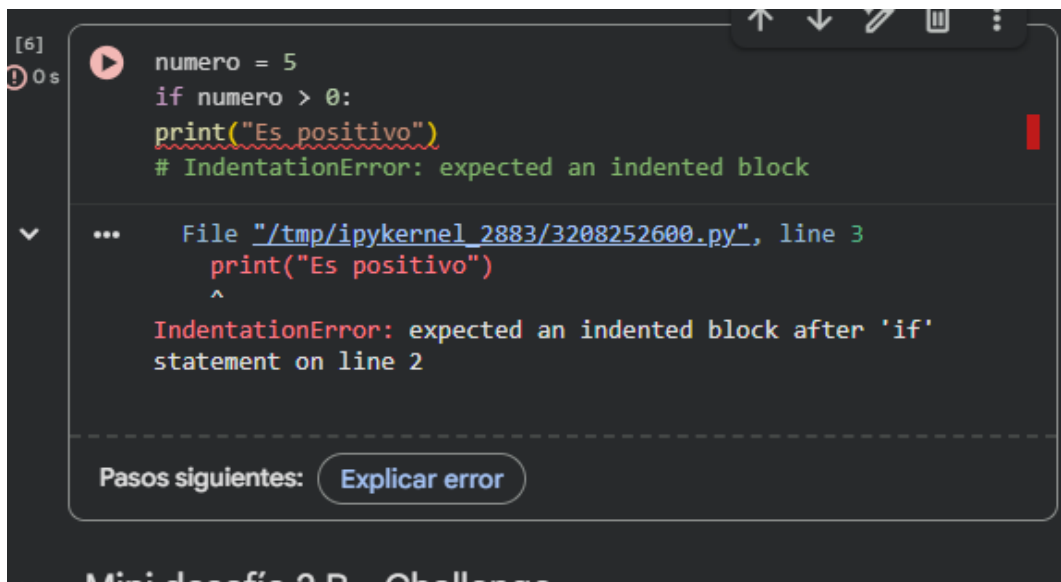
The screenshot shows a Jupyter Notebook cell with the following Python code:

```
[5]: numero = 5
    if numero > 0:
        print("Es positivo")
        print("Fin del bloque")
        print("Esto se ejecuta siempre")
```

The code is executed successfully, and the output is displayed below the code cell:

```
... Es positivo
    Fin del bloque
    Esto se ejecuta siempre
```

8.2 Otro tenga error por mala indentación.



The screenshot shows a Jupyter Notebook cell with the following Python code:

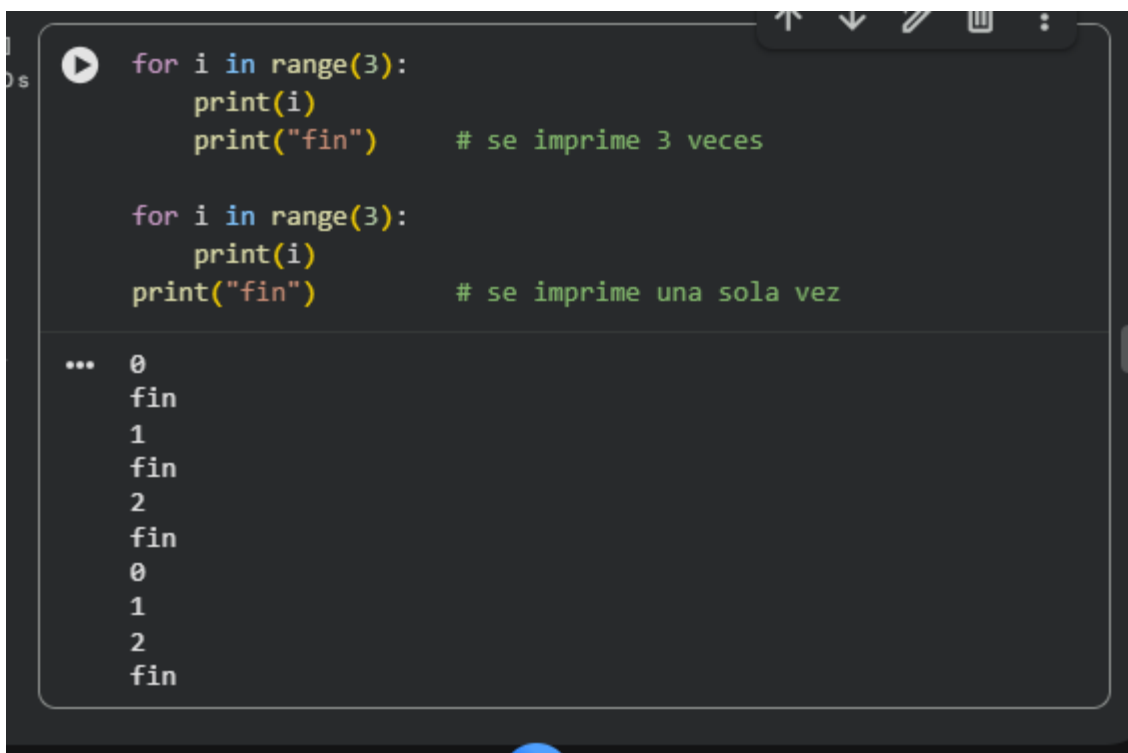
```
[6]: numero = 5
    if numero > 0:
        print("Es positivo")
        # IndentationError: expected an indented block
```

The code is executed, and an `IndentationError` is raised. The error message is displayed below the code cell:

```
... File "/tmp/ipykernel_2883/3208252600.py", line 3
      print("Es positivo")
      ^
IndentationError: expected an indented block after 'if'
statement on line 2
```

Below the error message, there is a section titled "Pasos siguientes:" with a button labeled "Explicar error".

Por qué importa la ubicación y no solo el error: en este ejemplo tenemos el mismo código con distinta indentación y se evidencia que cambia el resultado sin dar un error:



```
for i in range(3):
    print(i)
    print("fin")      # se imprime 3 veces

for i in range(3):
    print(i)
    print("fin")      # se imprime una sola vez
```

```
... 0
    fin
    1
    fin
    2
    fin
    0
    1
    2
    fin
```

9 Aplicaciones prácticas de Google Colab en proyectos reales.

Investiga y describe al menos tres usos en áreas como:

Ciencia de datos: nos permite cargar datos con pandas, por ejemplo, CSV de datos abiertos o de Kaggle, además podemos limpiarlos, analizarlos y graficarlos con la librería Matplotlib o Seaborn, y lo mejor es que cada paso queda documentado en el cuaderno, haciendo mucho más fácil replicar el análisis.

Inteligencia artificial y aprendizaje automático: es posible entrenar redes neuronales con TensorFlow o PyTorch aprovechando la GPU gratuita que ofrece este entorno, además cuanta con los tutoriales de TensorFlow y de Google como por ejemplo el Machine Learning Crash Course, y muchos ejemplos de Hugging Face se pueden abrir directamente en los cuadernos de Colab.

Educación: los docentes pueden compartir cuadernos con teoría y ejercicios listos para ejecutar y los estudiantes entregan tareas como enlaces y no tienen la necesidad de configurar nada.

Desarrollo de software: sirve para probar prototipos, el consumo de APIs, también para la automatización de tareas, el procesamiento de archivos, y muy importante, el poder validar ideas antes de llevarlas a un proyecto formal, aunque no sustituye a un entorno de desarrollo completo para aplicaciones grandes.

Investigación: muy práctico para compartir experimentos reproducibles con el código, los datos y las conclusiones, y lo mejor todo en un mismo sitio.

10 Reflexión crítica.

Explica cómo Google Colab puede ayudar a estudiantes que están aprendiendo Python y qué impacto tiene en el aprendizaje de programación actualmente.

Google Colab es un entorno que ayuda a reducir de manera significativa la percepción de complejidad que existe en torno al aprendizaje de la programación. Hace algunos años los estudiantes dejaban a un lado el proceso de aprendizaje apenas empezando, recursos como Colab aun no existían, entonces desde un inicio tenían que abordar problemas de instalación, versiones, o ineficiencia en el hardware.

Hoy en día con Colab solo basta tener un navegador y una cuenta en Google. Tiene un formato donde se fomenta el aprendizaje, y además su estructura permite que lo que se integre en una celda pueda ser probado y verificado de inmediato, siendo un método muy práctico ya que permite experimentar sin dañar o comprometer nada.

Aunque la finalidad del entorno es facilitar el desarrollo en sus múltiples ramas, se presta para mostrar de manera practica como entender los conceptos de compiladores. El estudiante puede ver todos los tipos de errores, léxicos, sintácticos y de indentación en vivo, facilitando identificar como es que Python interpreta el código que está escribiendo.

Es importante recalcar lo bueno que se vuelve cuando se deben desarrollar trabajos en equipo, también la cantidad de herramientas que trae consigo, y recalco dos que son de vanguardia a día de hoy, el acceso a la inteligencia artificial y a datos reales sin coste alguno, todo ello a un click en el navegador.

Aunque ofrece muchos beneficios, también posee puntos a considerar para su implementación como la necesidad de conexión a internet y obviamente tener cuenta el Google, los recursos gratuitos son mas orientados al campo del aprendizaje, es por ello que son limitados, y aunque la IA es un gran aliado su uso en exceso, ya sea en autocompletado o generación de código puede provocar la falta de comprensión en los procesos de aprendizaje.

La idea con este entorno es que se use como una herramienta de práctica, y aun así saber como maniobrar todo lo relacionado con Python u otro lenguaje de programación localmente.

11 Fuentes

<https://research.google.com/colaboratory/faq.html> ·

<https://colab.research.google.com/>

https://es.wikipedia.org/wiki/Google_Colab

<https://www.ironhack.com/es/blog/que-es-google-colab>